



Universitat de Girona



IFROS 2024-2026

Autonomous System

Lab2: Potential Function

Delivered by:

Pravin Oli (u1999699)
Gebrecherkos Gebreslassie (u1999542)

Supervisor:

Narcís Palomeras Rovira
Alaaeddine El Masri El Chaarani

Date of Submission:

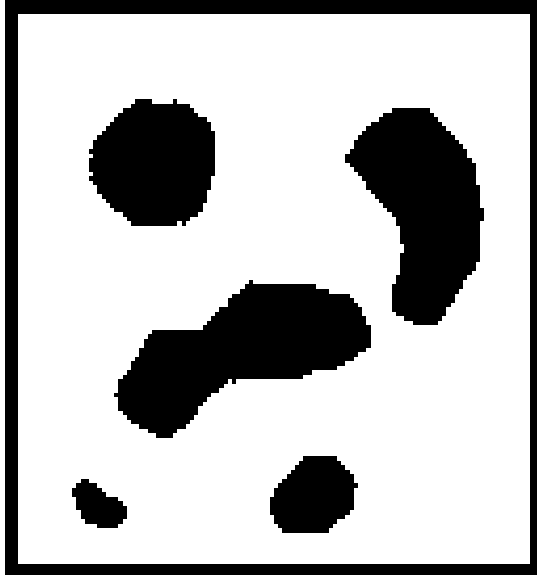
04/11/2024

Table of Contents

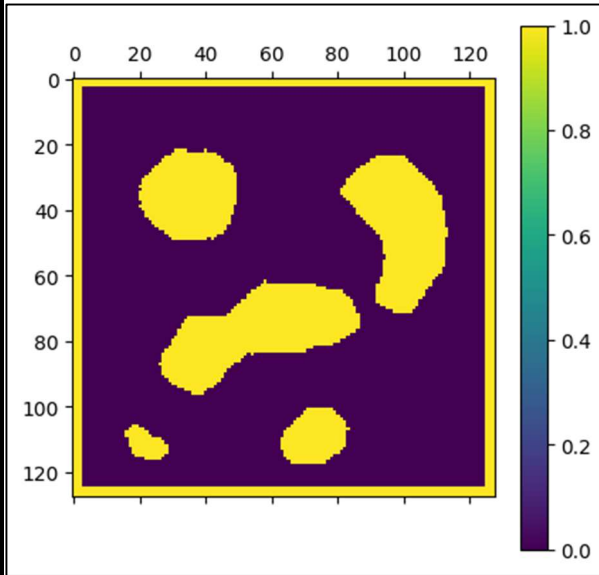
1.	Final Outcome:	3
1.1.	Part 1 (Potential Functions Outcome):	3
1.2.	Part 2 (Gradient Descent Outcome):	4
1.3.	Part 3 (Wave Font planner Outcome):	5
2.	Implementations:	7
2.1.	Part 1 (Potential Functions Implementation):	7
2.2.	Part 2 (Gradient Descent Implementation):	9
2.3.	Part 3 (Wave-Font planner Implementation):	9
3.	Challenges:	10
3.1.	Challenge in bushfire algorithm:	10
3.2.	Challenge in tuning attractive and Repulsive scale:	10
4.	Conclusion:	11

1. Final Outcome:

This lab consists of three parts. A set of grid map environments is provided in .png format for loading into a Python. In Part 1, the exercises include the Potential functions, Part 2 focuses on computing gradient and Part 3 addresses the wave-front planning algorithm.



Figure_1:Map Image .png



Figure_2 : Environment Map

1.1. Part 1 (Potential Functions Outcome):

The Part 1 result includes following:

a. Attraction Function

- Starting point: 10,10
- Goal point: 40, 110
- Result: Attraction potential is highest on the start and upper left corner (8000) and gradually decreases to Zero at the goal point as shown in Figure_3.

b. Repulsive Potential Function using Bushfire Algorithm

- It calculates the distance from each cell to nearest obstacle points.
- The result shows value is zero on obstacles and wall the map while maximum for the free space around the obstacles as shown in Figure_4.

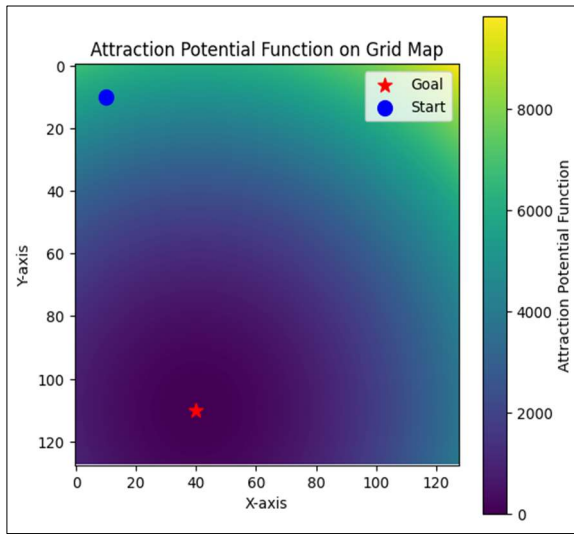
c. Repulsive Potential Function

- This function generates a repulsive force that push the robot away from obstacles, helping it avoid collisions as shown in Figure_5.
- Q is a scaling factor that determines the strength of the repulsive potential. Higher values of Q , increase the repulsive force.
- The result shows that the repelling force is very high near to the obstacles and decreases at farther distance.

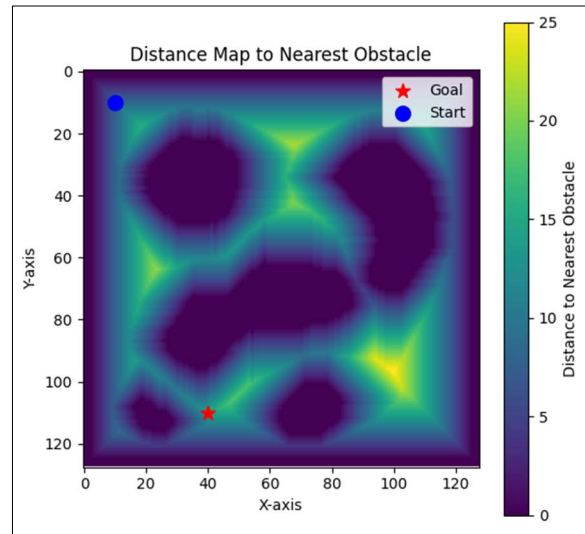
d. Total Potential Function

- It combines attractive and repulsive to form a path planning strategy.
- Robot is attracted towards goal and repelled away from obstacles

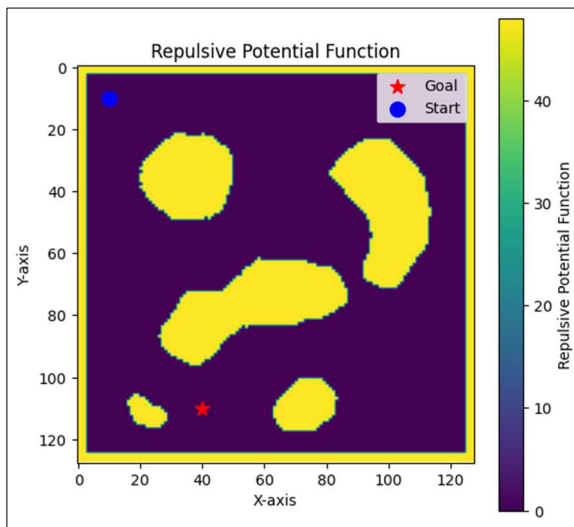
- The result shows infinite repulsion at obstacles and boundary, minimum attraction near the goal and maximum attraction near the starting point and far away from goal as shown in Figure_6.



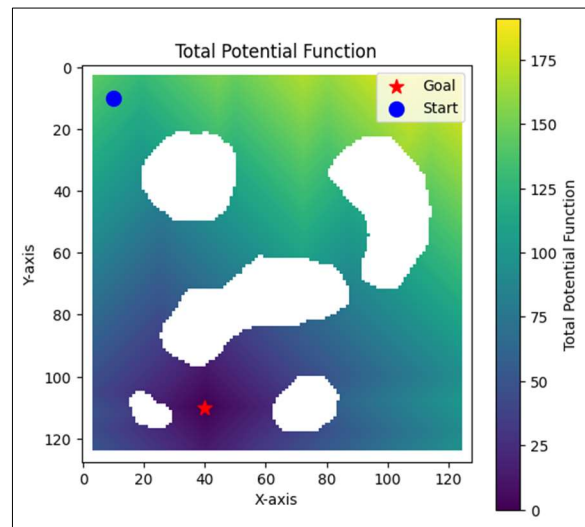
Figure_3: Attraction function



Figure_4: Distance map to nearest obstacles



Figure_5: Repulsive potential function



Figure_6: Total potential function

1.2. Part 2 (Gradient Descent Outcome):

- This Part is about calculation of robot path following the negative gradient to reach the goal. We used connectivity 4 algorithm to compute the gradient for a particular cell.
- The difference between the potential value of a cell and its directly connected neighbors (up, down, left, right). This process is repeated iteratively, ideally reaching a local or global minimum.

The basic characteristics of the code are:

- Max iteration step = 10000
- Tolerance value = 10-4
- Tentative Run time = approx. 0.4 s
- Method used: 4-point connectivity

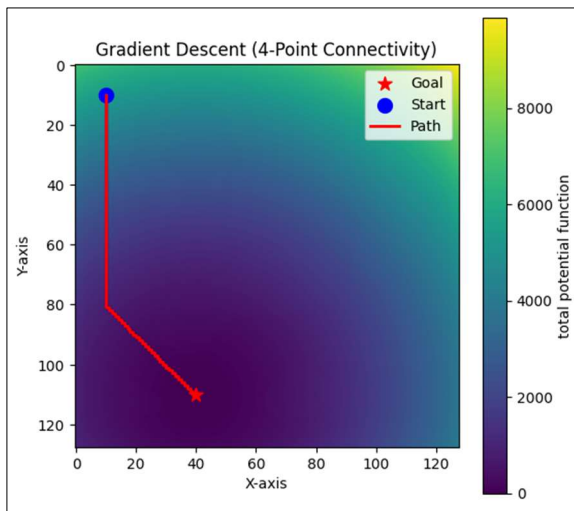
- e. Directions = $[(-1, 0), (1, 0), (0, -1), (0, 1)]$ (up, down, left, right)

Question 1: To test it, use map0.png with the start position at (10, 10) and the goal position at (110, 40) first and (90, 70) later. What happens? Why?

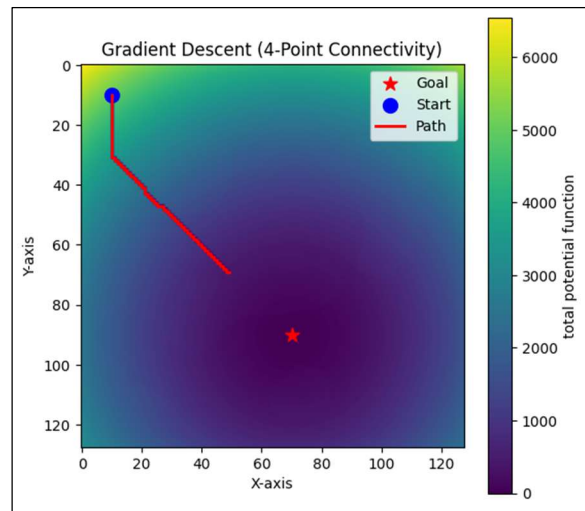
Answer: *For Goal Position (y=110, x=40):* The trajectory of path from start converges continuously avoiding the obstacles and reaches on global minimum at goal as shown in Figure_7.

For Goal Position (y=90, x=70): The trajectory of path from start becomes discontinuous, as it gets trapped in a local minimum as shown in Figure_8.

Conclusion: In cases where local minima exist along the path, the algorithm might get trapped, failing to reach the global minimum or taking a longer, interrupted path.



Figure_7: for goal (y=110, x=40)



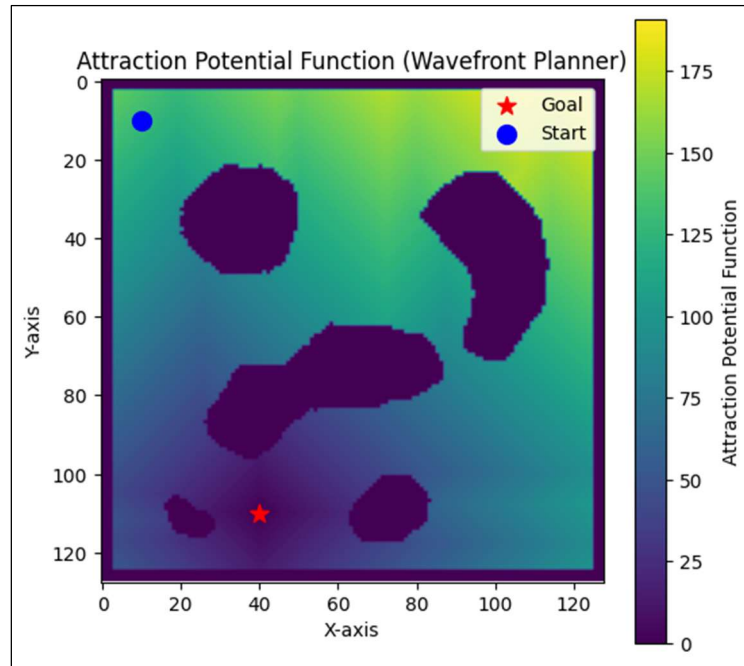
Figure_8: for goal (y=90, x=70)

1.3. Part 3 (Wave Font planner Outcome):

- This Part is about calculation of robot path using Wave font planner function, by expanding a wave-like front from the goal, propagating through free space, and labeling cells with distance values.

The basic characteristics of the code are:

- a. Initially attraction potential for grid = Infinity
- b. Initially attraction potential for goal = 0
- c. Tentative Run time = approx. 0.5 s
- d. Method used: 4-point connectivity
- e. Directions = $[(-1, 0), (1, 0), (0, -1), (0, 1)]$ (up, down, left, right)

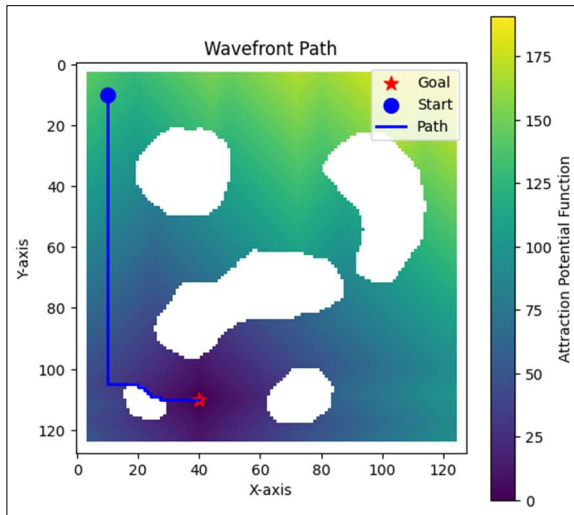


Figure_9: Attraction potential function (wave front planner)

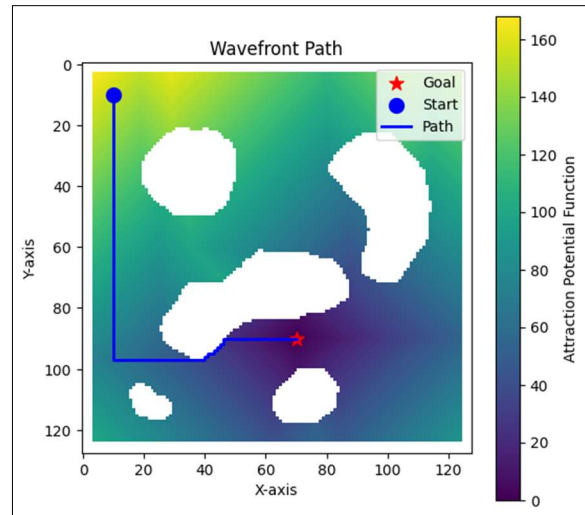
Question 2: To test it, use map0.png with the start position at (10, 10) and the goal position at (110, 40) first and (90, 70) later. What happens? Why?

Answer: For Goal Position ($y=110$, $x=40$) and Goal Position ($y=90$, $x=70$), the trajectory of path from start converges continuously avoiding the obstacles and reaches on global minimum at goal as shown in Figure_10 and Figure_11.

Conclusion: In any cases the wave font planner function always converges in global minimum value unlike bushfire algorithm.



Figure_10: for goal ($y=110$, $x=40$)



Figure_11: for goal ($y=90$, $x=70$)

2. Implementations:

This section describes the implementation of potential functions using bush fire algorithm and wave-front planner in python code:

2.1. Part 1 (Potential Functions Implementation):

a. Attraction Function

The implementation step for this part is listed as:

- *Function Definition:* compute_attraction_potential
- *Inputs:* Takes grid_map, start_position, and goal_position as parameters.
- *Grid Dimensions:* Extracts the height and width of the grid_map.
- *Scale Factor:* Defines a scale factor (set to 1) for calculating the potential.
- *Potential Calculation:* Uses a quadratic formula for each cell.
- *Output:* Returns the computed attraction potential field.
- *Positions:* Defines start_position and goal_position.
- *Plotting:*

$$U_{att}(q) = \frac{1}{2} \zeta d^2(q, q_{goal})$$

```
for y in range(height):
    for x in range(width):
        attraction_potential [y, x] = 0.5*scale*(np. Sqrt ((y -goal_position [0])
**2 + (x - goal_position [1]) **2)) |**2
return attraction_potential
```

b. Repulsive Potential Function using Bushfire Algorithm

- *Define Function:* brushfire_algorithm.
- *Initialize Distance* Queue and Obstacles Map:
- *Define Directions:* four directions (N, S, E, W).
- *BFS Calculation:* Perform Breadth first search to update distances:
- *Dequeue cell*, check neighbors, update distances if shorter.
- *Enqueue* updated cells.
- *Return Result:* Return the computed distance_map.
- *Plot Results:* Visualize the distance_map

$$U_{rep}(q) = \begin{cases} \frac{1}{2} \eta \left(\frac{1}{D(q)} - \frac{1}{Q^*} \right)^2, & D(q) \leq Q^* \\ 0, & D(q) > Q^* \end{cases}$$

```
# Perform BFS to calculate distances
while queue:
    y, x = queue.popleft()
    current_distance = distance_map[y, x]

    # Check all 4 directions
    for dy, dx in directions:
        ny, nx = y + dy, x + dx
        if 0 <= ny < height and 0 <= nx < width:
            if distance_map[ny, nx] > current_distance + 1:
                distance_map[ny, nx] = current_distance + 1
                queue.append((ny, nx))

    return distance_map

distance_map = brushfire_algorithm(grid_map)
```

c. Repulsive Potential Function

- *Function:* compute_repulsive_potential
- *Initialize:* Create a zero-filled repulsive_potential array.
- *Loop:* Iterate through each cell in the distance_map.
- *Compute Potential:* If distance < Q, calculate repulsive potential, Otherwise, set to 0.
- *Return:* Output the computed repulsive_potential.
- *Set Q:* Define scaling factor Q.
- *Visualization:* Plot the repulsive potential

```
for y in range(distance_map.shape[0]):
    for x in range(distance_map.shape[1]):
        d = distance_map[y, x]
        if d < Q:
            repulsive_potential[y, x] = 0.5 * (1/(d + epsilon) - 1/Q) ** 2
        else:
            repulsive_potential[y, x] = 0

return repulsive_potential
```

d. Total Potential Function

- *Define Function:* compute_total_potential
- *Calculate Total Potential:* Sum the attraction and repulsive potentials
- *Return Total:* Output the computed total_potential.
- *Plot Results:* Visualize the total potential

$$U(q) = U_{att}(q) + U_{rep}(q)$$

```
total_potential = attraction_potential + repulsive_potential

return total_potential

total_potential = compute_total_potential(attraction_potential, repulsive_potential)
```

2.2. Part 2 (Gradient Descent Implementation):

- *Function:* Define `gradient_descent_4connectivity` for pathfinding using gradient descent.
- *Initialize:* Start with the initial position in the path list.
- *Define Directions:* Use 4-point connectivity (up, down, left, right).
- *Loop:* Check if near the goal, Identify and filter valid neighbors based on potential and Move to the neighbor with the lowest potential.
- *Stop Conditions:* Exit if no lower potential neighbors are found and Update position
- *Visualize:* Mark the path on a copy of the total potential map.
- *Return:* Output the path and modified map.
- *Plot:* Display total potential

```
# Gradient descent with 4-point connectivity
for _ in range(max_steps):
    y, x = current_position

    # if close enough to the goal
    if np.linalg.norm(np.array([y, x]) - np.array(goal)) < tolerance:
        print("Goal reached!")
        break

    # Neighbors cells in the 4-point connectivity
    neighbors = [(y + dy, x + dx) for dy, dx in directions]
    valid_neighbors = [(ny, nx) for ny, nx in neighbors if 0 <= ny < total_potential.shape[0] and 0 <= nx < total_potential.shape[1]]

    # Neighbor cell with the lowest potential value
    next_position = min(valid_neighbors, key=lambda pos: total_potential[pos[0], pos[1]])

    # If no lower potential, terminate
    if total_potential[next_position[0], next_position[1]] >= total_potential[y, x]:
        print("Reached local minimum, stopping search.")
        break

    # Move to the next cell
    current_position = next_position
    path.append(current_position)

# Check for goal position within tolerance
if current_position == goal:
    print("Goal reached!")
    break
```

2.3. Part 3 (Wave-Font planner Implementation):

- *Function Creation:* `wave_front_planner` to compute attraction potential.
- *Initialize Field:* Set a matrix to high values, with the goal position at 0 and obstacles at 0.
- *BFS Queue:* Use a queue starting with the goal position.
- *Directions:* Define movements using 4-point connectivity (up, down, left, right).
- *Propagation Logic:* Update neighbors' potentials if the new value is lower, enqueue valid neighbors.
- *Load Map:* Convert a binary image into a grid map (0 for free, 1 for obstacles).
- *Specify Positions:* Define start and goal coordinates.

- *Compute Potential:* Call the planner function with the grid map and positions.
- *Visualization:* Plot the attraction potential field.

```
rows, cols = grid_map.shape
attraction_potential = np.full((rows, cols), np.inf) # Starting with infinity
attraction_potential[goal_position] = 0 # Setting the goal position to 0
attraction_potential[grid_map == 1] = 0 # Setting potential for obstacles to 0

# Queue for BFS
queue = deque([goal_position])

# Define the 4 connectivity directions (up, down, left, right)
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

# Perform wavefront propagation
while queue:
    current = queue.popleft()
    current_y, current_x = current

    # Process neighbors
    for dy, dx in directions:
        neighbor_y, neighbor_x = current_y + dy, current_x + dx

        # Check if the neighbor is within bounds and not an obstacle
        if 0 <= neighbor_y < rows and 0 <= neighbor_x < cols and grid_map[neighbor_y, neighbor_x] == 0:
            # Calculate potential value based on distance from current position
            new_potential = attraction_potential[current_y, current_x] + 1
            if new_potential < attraction_potential[neighbor_y, neighbor_x]:
                attraction_potential[neighbor_y, neighbor_x] = new_potential
                queue.append((neighbor_y, neighbor_x)) # Add the neighbor to the queue

return attraction_potential
```

3. Challenges:

3.1. Challenge in bushfire algorithm:

Problem: Accurately computing the distance from each cell to the nearest obstacle on the grid.

Solution: Implemented a breadth-first search (BFS) approach to propagate distances from obstacle cells throughout the grid, initializing a distance map with infinity and updating it based on neighboring cells.

3.2. Challenge in tuning attractive and Repulsive scale:

Problem: Accurately choosing scale factor for attractive and Q for repulsive.

Solution: I choose Scale factor= 1 and Q=5. which means moderate pull towards goal and high push from obstacles for avoiding collision.

4. Conclusion:

In this lab, we implemented potential function methods to help a robot navigate a grid-based map from a starting to a goal avoiding obstacles. Using total potential and gradient descent algorithms, we tracked the robot's paths to both global and local minima and saw how changing the goal position affected these paths. We also found that the wavefront planner reliably reached global minima, demonstrating its effectiveness for pathfinding.

The Gradient descent approach may struggle with local minima but can be effective in well-structured environment and is faster in finding path in simple environment. The Wave-front planner doesn't stock on local minima and can be used in any dynamic environment. It reliably reaches global minima and effectively avoids obstacles.